accautomation.ca

Building a PLC Program That You Can Be Proud Of – Part 1

6-8 minutes

What is the best way to program a PLC?

My answer is simple. The best way is one in which someone can look at your program and understand it. I cannot stress enough the need for good documentation of your program. The best programs are ones that I can return to after several years and understand what it is doing within a few minutes. Programs should read like a book. This will aid in troubleshooting, modifying or teaching.

How do you approach a PLC program?

You must know everything about the logic or process before starting your program. Making a flow chart is one good method to learning the logic and process. The flow chart will bring out questions like the following:

What happens after a power outage? (In each condition of the outputs)

What happens if a sensor is not made? How long do you wait?

What are the critical items to monitor? (Ex. Air Pressure, Weight, Length, etc)

What happens…

Once you have written your program and are in the troubleshooting stage you can usually go back and add to your flow chart. Usually

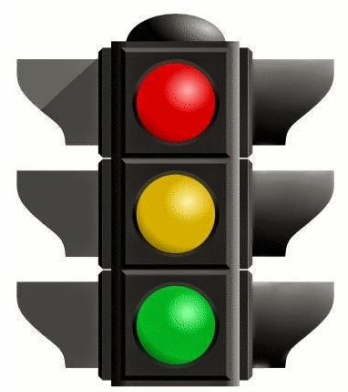
there is always something that needs to be added, changed or modified based upon the actual functioning of the program. Consider each project a complete leaning opportunity.

Once you know what you want to do with the PLC and have a good understanding of the logic flow, then it is time to start coding. Remember that there is no write or wrong method to program the PLC, either the program will work or it will not work.

Let's look at an example. We will start with something that we all know how it works.

Traffic Lights

We will look at three programming examples for the lights. Two different approaches to programming will be used, but the program function is the same. The last example will modify the logic for a car being sensed.



Logic:

First Example:

Traffic Light Program

Sample program for traffic light intersection with lights facing North /South and West /East.

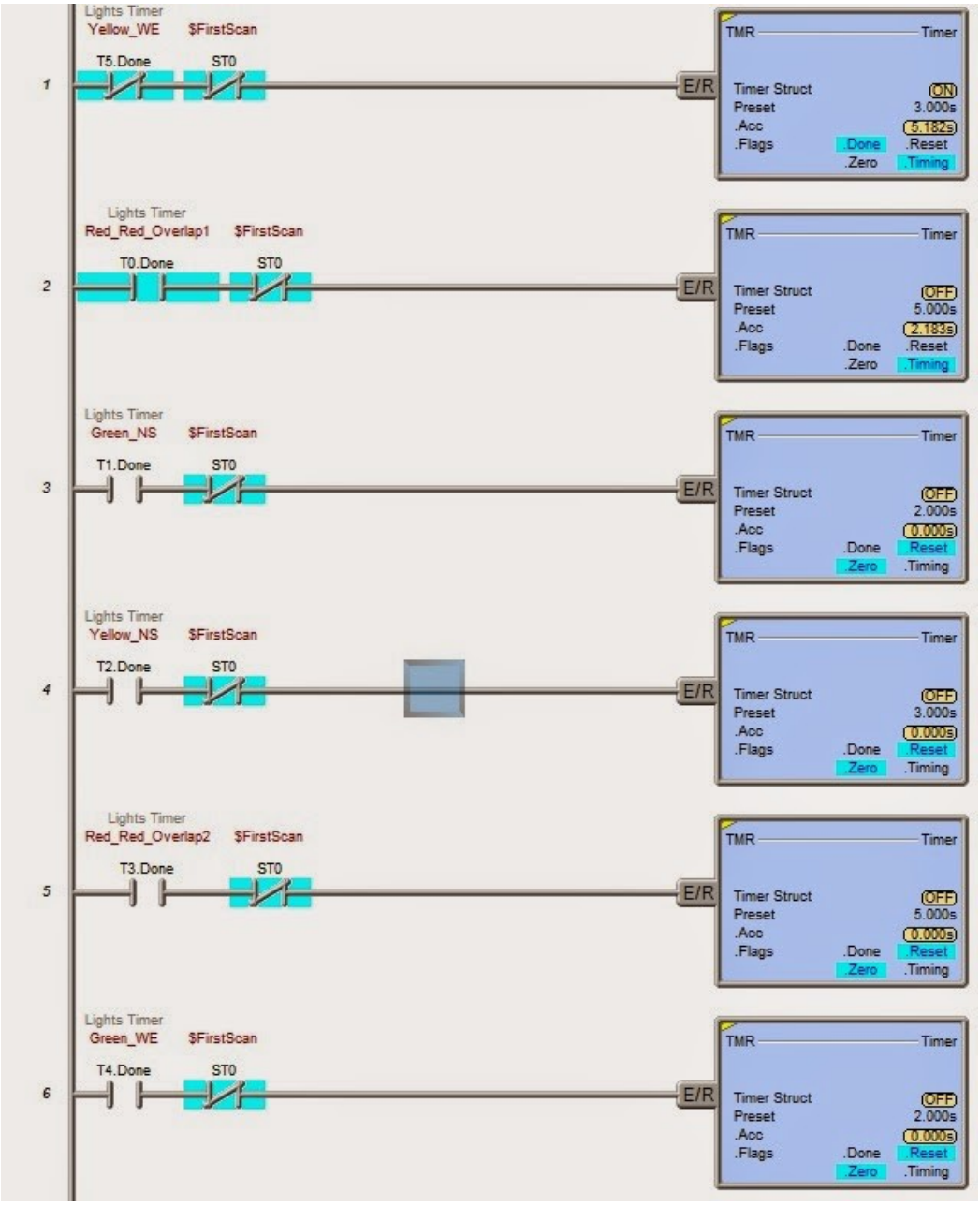
Green is on for 5 seconds

Yellow is on for 2 seconds

Red has an overlap of 3 seconds

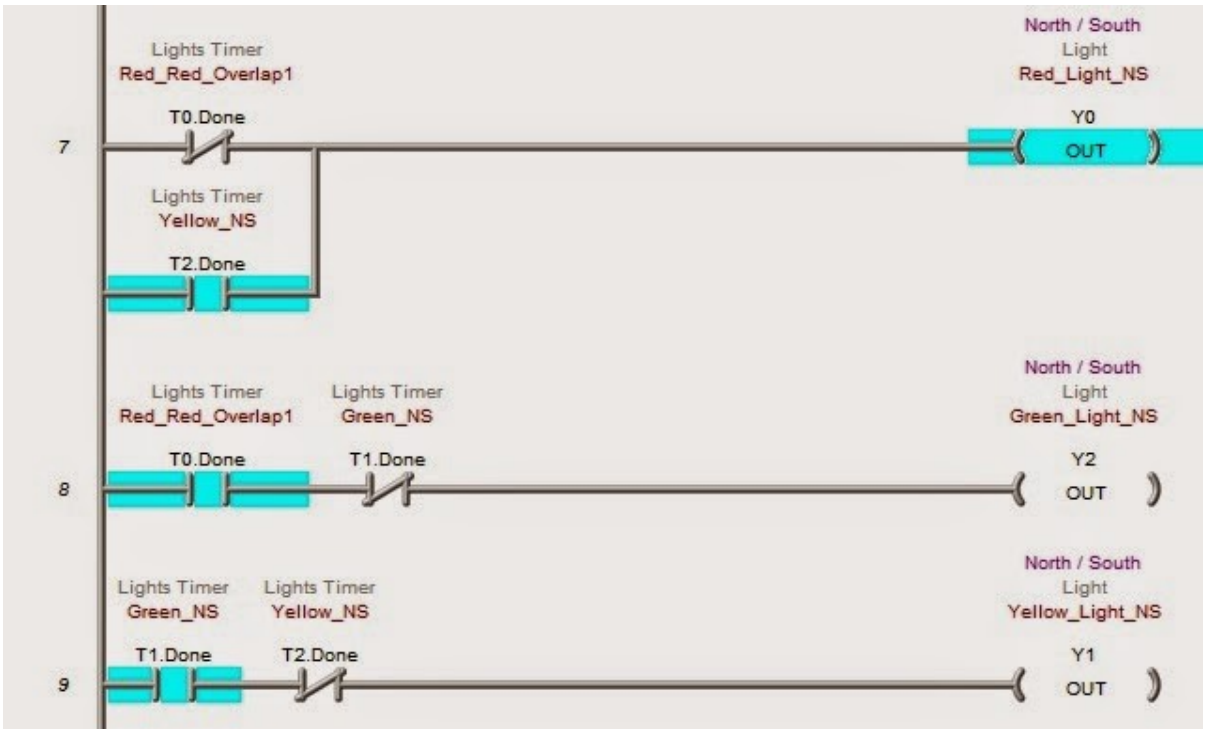
This program uses discrete bits and timers to accomplish this task.

The $FirstScan bit will reset the timers so if power is lost, the lights will start with Red / Red overlap before starting the sequence again.

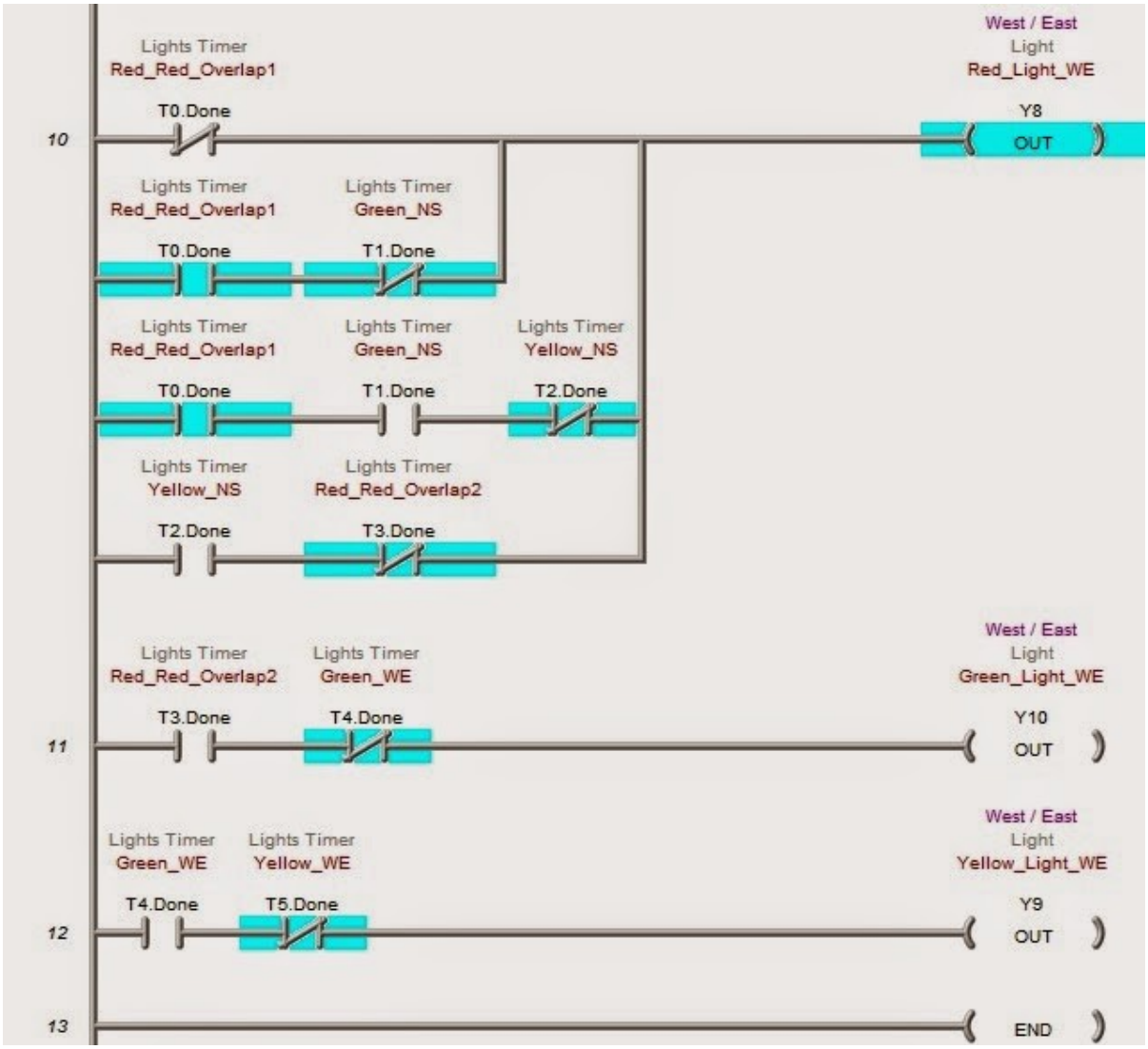


The outputs are controlled by when the timers are on (Done) or off (Not Done)

North / South Traffic Lights



West / East Traffic Lights



You will notice that this program is fully documented and easy to understand.

This program is based upon time events. The base rate is one second. If we create a list of what the outputs look like after each second and then send them to the physical output channel we will have the second type of approach to this logic…

Logic:

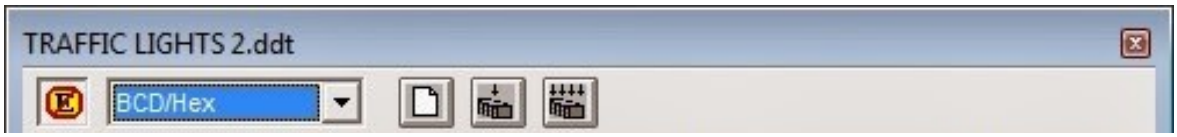
Second Example:

Traffic Light Program

Sample program for traffic light intersection with lights facing North /South and West /East.
Green is on for 5 seconds
Yellow is on for 2 seconds
Red has an overlap of 3 seconds

This program uses indirect addressing to program
Lets look at the list of outputs we want based upon the following addresses: (Notice the Bit location)
Y0 – Red_Light_NS
Y1 – Yellow_Light_NS
Y2 – Green_Light NS
Y8 – Red_Light_WE
Y9 – Yellow_Light_WE
Y10 – Green_Light_WE

We have 20 steps to do in the sequence based upon 1 second increments. (V1000 to V1019)



	Element	Status	Edits
1	V0	1017	1000
2	V1	0000_0100_0000_0001	
3	V1000	0x0101	101
4	V1001	0x0101	101
5	V1002	0x0101	101
6	V1003	0x0104	104
7	V1004	0x0104	104
8	V1005	0x0104	104
9	V1006	0x0104	104
10	V1007	0x0104	104
11	V1008	0x0102	102
12	V1009	0x0102	102
13	V1010	0x0101	101
14	V1011	0x0101	101
15	V1012	0x0101	101
16	V1013	0x0401	401
17	V1014	0x0401	401
18	V1015	0x0401	401
19	V1016	0x0401	401
20	V1017	0x0401	401
21	V1018	0x0201	201
22	V1019	0x0201	201

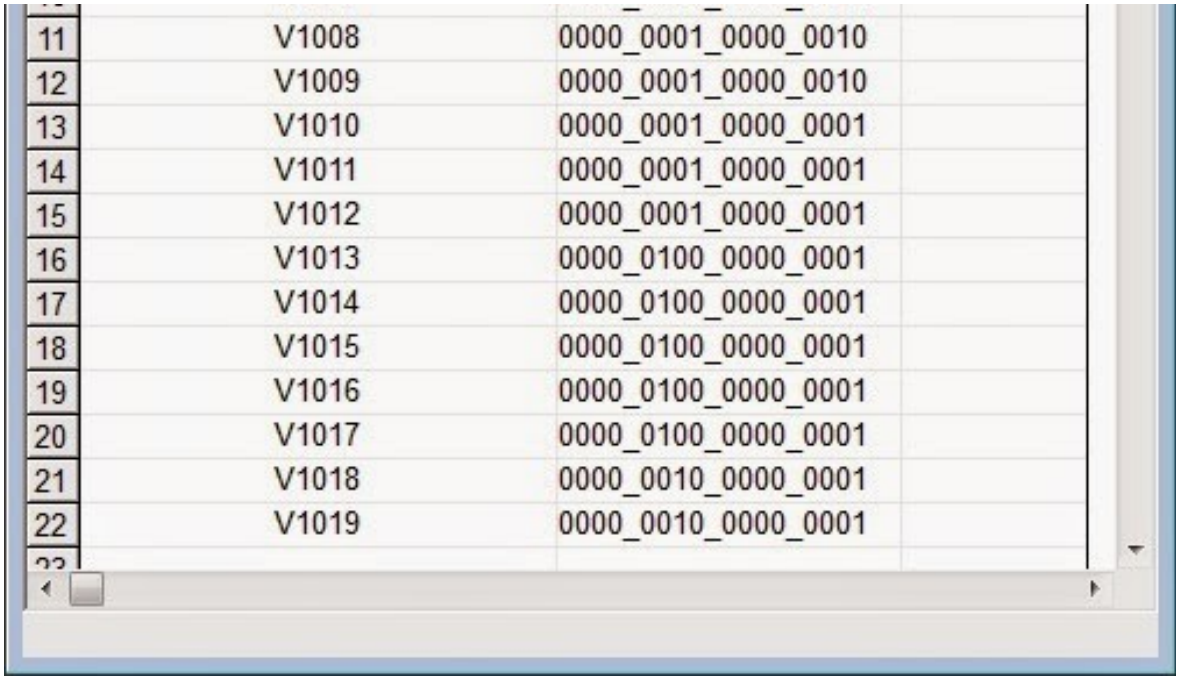
Here is what the hex values translated to binary look like:

(Review of numbering systems from previous blog)

TRAFFIC LIGHTS 2.ddt

Binary

	Element	Status	Edits
1	V0	1013	
2	V1	0000_0100_0000_0001	
3	V1000	0000_0001_0000_0001	
4	V1001	0000_0001_0000_0001	
5	V1002	0000_0001_0000_0001	
6	V1003	0000_0001_0000_0100	
7	V1004	0000_0001_0000_0100	
8	V1005	0000_0001_0000_0100	
9	V1006	0000_0001_0000_0100	
10	V1007	0000_0001_0000_0100	

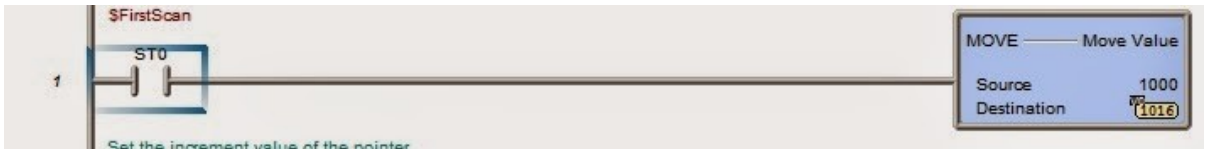


11	V1008	0000_0001_0000_0010
12	V1009	0000_0001_0000_0010
13	V1010	0000_0001_0000_0001
14	V1011	0000_0001_0000_0001
15	V1012	0000_0001_0000_0001
16	V1013	0000_0100_0000_0001
17	V1014	0000_0100_0000_0001
18	V1015	0000_0100_0000_0001
19	V1016	0000_0100_0000_0001
20	V1017	0000_0100_0000_0001
21	V1018	0000_0010_0000_0001
22	V1019	0000_0010_0000_0001

The $FirstScan bit will reset the pointers so if power is lost, the lights will start with Red / Red overlap before starting the sequence again.

Lets look at the program:

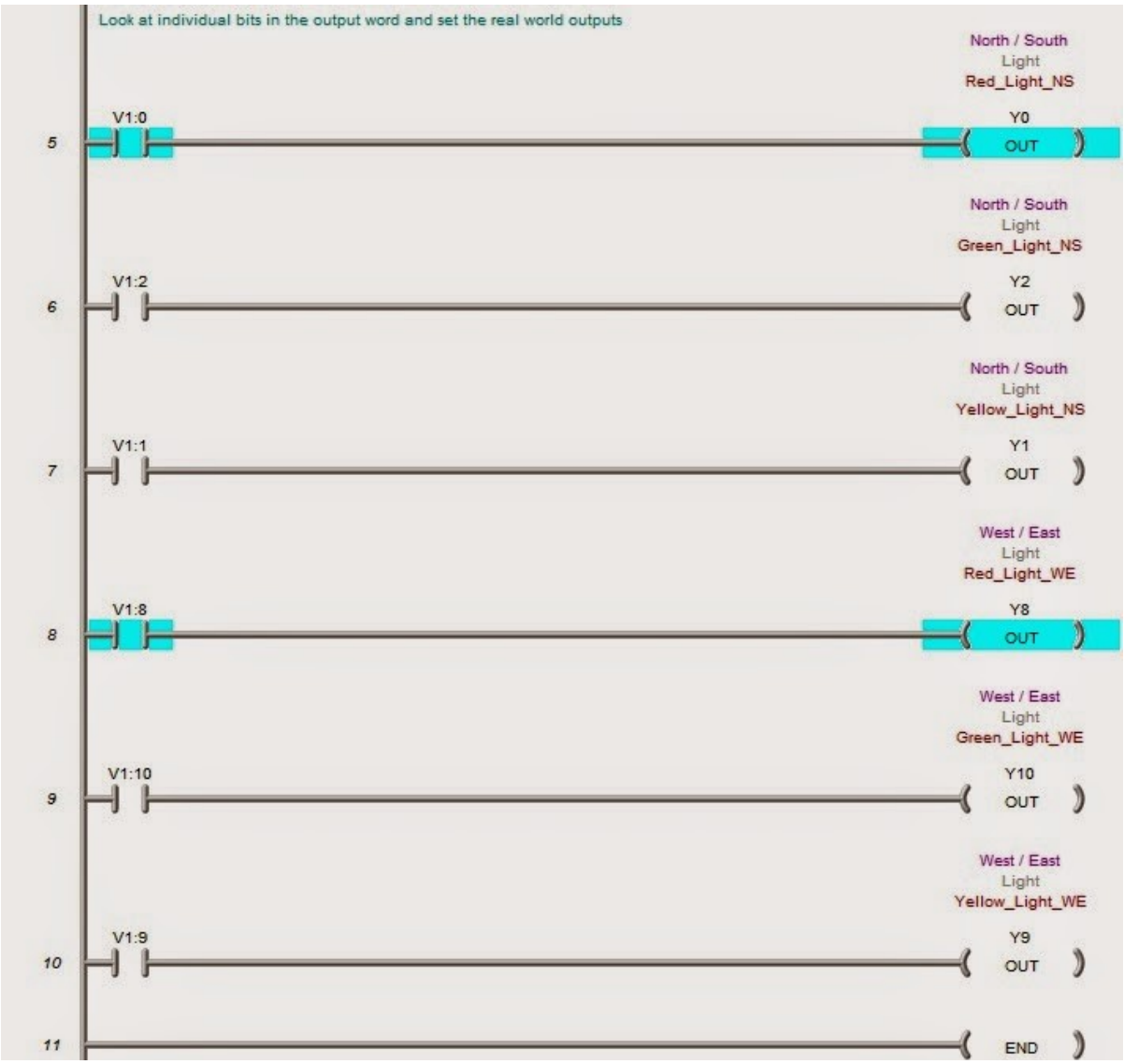
The $FirstScan bit will move the number 1000 into V0. V0 will act as our pointer for the list of outputs. (V1000 to V1019)
Every 1 second ($1Second) V0 will increment by a value of 1. We will then compare the value to 1020 which indicates the end of the sequence. If the value is greater or equal to then our pointer is reset to the value of 1000. This is done by moving the number 1000 into V0.
The last step is moving our output word indirectly V0 to our output word V1. Indirectly means that the value in V0 will point to a memory location dictated by the number it contains.
ex: V0 has a value of 1000 so this means that V[V0] will move V1000 into our output word.





Set the outputs

Our physical outputs are set by casing our output word (V1) into bits. Depending on the programmable controller this can be done my moving to a word that can be addressed by bits or in our case we can cast the word into bits. [V1:#]

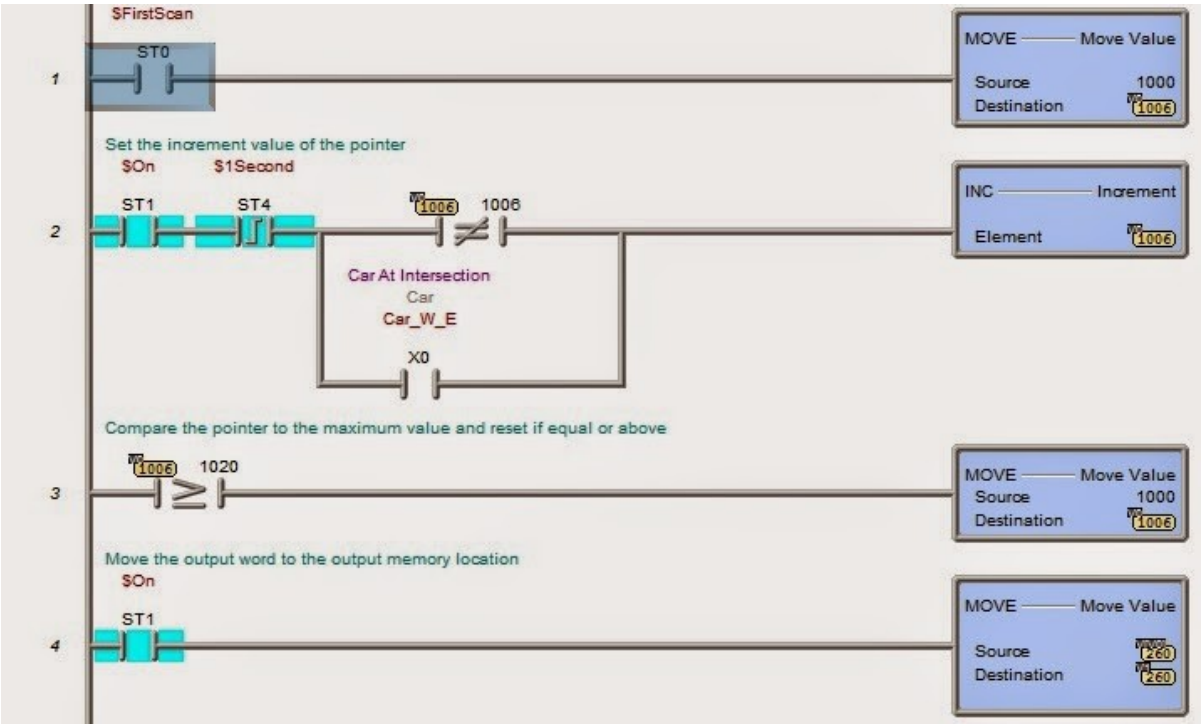


This program code is allot smaller than the first using discrete bits and timers. With documentation it is also easier to read.

One of the advantages of indirect addressing to program is that it makes modifications easier. Lets modify the last program…

The North will stay green until a car approaches from the West. It will remain green for 1 more second before turning yellow and completing the cycle. If the car is always there then the lights will always function.

X0 – Car West/East



Just a couple of contacts have been added to the logic on the line that increments the pointer. The setting of the outputs do not change.

If the value at V0 is equal to 1006 then stop incrementing V0. X0 (Car at intersection) comes then the pointer will increment. The cycle will complete and continue until X0 is not present. It will then stop when the pointer V0 equals 1006 again.

Watch on YouTube : Building a PLC Program That You Can Be Proud Of

In part 2 we will look at indirect addressing with a sequence that is event driven, not timed like the above.

Contact me for the above programs. I will be happy to email them to you.
If you have any questions or need further information please contact me.
Thank you,
Garry

You can download the software and simulator free at the following address. Also listed are helpful guides to walk you through your first program.
Do-more Designer Software

How to use video's for Do-more Designer Software

One of the better PLC programming books is PLC Programming for Industrial Automation by Keven Collins. Here is the link to the free download.

If you're like most of my readers, you're committed to learning about technology. Numbering systems used in PLC's are not difficult to learn and understand. We will walk through the numbering systems used in PLCs. This includes Bits, Decimal, Hexadecimal, ASCII and Floating Point.

To get this free article, subscribe to my free email newsletter.

Use the information to inform other people how numbering systems work. Sign up now.



The ‘Robust Data Logging for Free’ eBook is also available as a free download. The link is included when you **subscribe to ACC Automation**.